Algoritmi

Marco Pittarello

Contents

1 Insertion Sort **2**
1.1 Pseudocodice . . . 2
1.2 Analisi . . . 2
1.3 Casi . . . 3

2 Merge Sort **3**
2.1 Pseudocodice . . . 3
2.2 Correttezza . . . 3
2.3 Analisi . . . 4
2.3.1 Albero di ricorsione . . . 4
2.4 Confronto Insertion Sort vs Merge Sort . . . 4

3 Principio di induzione **4**
3.1 Induzione forte . . . 5
3.2 Alberi binari . . . 5

4 Tempo di esecuzione **5**
4.1 Metodo del limite . . . 5

5 Complessità dei problemi **6**

6 Metodo di sostituzione **6**

7 Master Theorem **7**
7.1 Utilizzo . . . 7

8 Insertion Sort Ricorsivo **8**
8.1 Pseudocodice . . . 8
8.2 Analisi . . . 8
8.3 Duplicazione . . . 8

9 HeapSort **9**

10 QuickSort **9**

11 Esercizi **10**

Definizione 1: Algoritmo

Procedura che con semplici istruzioni descrive come risolvere un problema.

Di ogni algoritmo bisogna sapere:

- Correttezza
- Stabilità
- Complessità

1 Insertion Sort

Info 1: Idea

Si costruisce incrementalmente un sottoarray ordinato `A[1..j-1]`. A ogni passo si inserisce `A[i]` nella posizione corretta.

animazione

1.1 Pseudocodice

```
InsertionSort(A)      #c0
    m = A.length      #c1
    for j = 2 to m    #nc2
        key = A[j]    #(n-1)c3
        i = j - 1     #(n-1)c4
        while i > 0 && A[i] > key # sum (t_j+1)c5
            A[i + 1] = A[i]       # sum t_j c6
            i = i - 1             # sum t_j c7
        A[i + 1] = key            #(n-1)c8
```

Tutte le operazioni elementari (assegnazione, operazioni aritmetiche) hanno costo costante. I cicli invece hanno il costo di una sommatoria $\sum$.

1.2 Analisi

Efficiente per piccole quantità di dati, è uno degli algoritmi più semplici.

- Modello di calcolo: operazioni elementari a costo costante.
- Dimensione del problema: numero di elementi n.

$$T^{IS}(n) = c_0 + c_1 + nc_2 + (c_3 + c_4 + c_8)(n-1) + \sum_{j=2}^{n}(t_j + 1)c_5 + \sum_{j=2}^{n} t_j(c_6 + c_7)$$

dove t_j è il numero di iterazioni del `while` per l'indice j.

Attenzione 1: Problemi

- Troppe costanti ignote.
- Dipende dalla specifica istanza del problema.

1.3 Casi

Caso	Condizione	t_j	Complessità
Migliore	array già ordinato	0	$T_{IS}^{min}(n) = an + b = \Theta(n)$
Peggiore	array ordinato al contrario	$j-1$	$T_{IS}^{max}(n) = an^2 + bn + c = \Theta(n^2)$
Medio	mediamente	$\frac{j-1}{2}$	$\Theta(n^2)$

Osservazione: solitamente il caso medio coincide con quello peggiore. Dato che ci sono 2 cicli il caso peggiore e quello medio avrà complessità n^2.

2 Merge Sort

Info 2: Divide et Impera

- **Divide**: divide il problema in sottoproblemi.
- **Impera**: risolve ricorsivamente i sottoproblemi.
- **Combina**: fonde le soluzioni in una soluzione del problema originale.

animazione

2.1 Pseudocodice

```
MergeSort(A, p, r)
    if p < r
        q = (p + r)/2
        MergeSort(A, p, q)
        MergeSort(A, q + 1, r)
        Merge(A, p, q, r)

Merge(A, p, q, r)   // Pre: A[p..q] e A[q+1..r] sono ordinati.
    n1 = q - p + 1  // Post: A[p..r] è ordinato.
    n2 = r - q

    for i = 1 to n1
        L[i] = A[p + i - 1]
    for j = 1 to n2
        R[j] = A[q + j]

    L[n1 + 1] = +∞
    R[n2 + 1] = +∞

    i = j = 1
    for k = p to r
        if L[i] <= R[j]
            A[k] = L[i]
            i++
        else
            A[k] = R[j]
            j++
```

2.2 Correttezza

- Si dimostra per induzione su $l = r - p$.
- **Base** $l = 0$: `p = r`, `A[p..p]` un solo elemento $\Rightarrow$ già ordinato.
- **Passo** $l > 0$ $(p < r)$: si divide `A[p..r]` in due parti. Per ipotesi induttiva, `MergeSort` ordina `A[p..q]` e

`A[q+1..r]`. `Merge` opera sui due sottoarray ordinati e, soddisfatta la precondizione, produce `A[p..r]` ordinato.

2.3 Analisi

- Spazio occupato in memoria: $a \cdot n + b$
  a = spazio occupato da un elemento dell'array (`int` 4 byte)
  n = numero di elementi dell'array
  b = spazio occupato dagli *elementi di controllo* (`if`, `for`)

Ricorrenza:

$$T^{MS}(n) = \begin{cases} c_0 & n \leq 1 \\ T^{MS}\left(\lfloor \frac{n}{2} \rfloor\right) + T^{MS}\left(\lceil \frac{n}{2} \rceil\right) + an + b & n > 1 \end{cases}$$

2.3.1 Albero di ricorsione

- altezza: $h = \log_2 m$
- costo a livello j: $am + 2^j b$
- numero foglie: mc_0

$$\begin{aligned} T^{MS}(n) &= \left(\sum_{j=0}^{h-1} an + 2^j b \right) + nc_0 \\ &= ahn + \left(2^h - 1\right) b + nc_0 \\ &= an \log_2 n + (n-1)b + nc_0 \\ &\approx \Theta(n \log n) \end{aligned}$$

2.4 Confronto Insertion Sort vs Merge Sort

Suggerimento 1: Interpretazione

Per n grandi, MergeSort è asintoticamente migliore.
Per n piccoli, InsertionSort può essere più veloce per via delle costanti inferiori.

$$\begin{aligned} T^{IS}(n) &\approx n^2 \\ T^{MS}(n) &\approx n \log n \end{aligned}$$

3 Principio di induzione

Per dimostrare che $P(m)$ vale $\forall\ m \in \mathbb{N}$:

1. **Base**: dimostro $P(0)$.
2. **Passo induttivo**: assumo $P(m)$ e dimostro $P(m+1)$.

Allora $P(m)$ vale $\forall\ m$.

Esempio 1: Principio di induzione

$Q(m):\ 2^m \leq (m+1)!$
- Base $Q(0)$: $2^0 = 1 \leq (0+1)! = 1$.
- Passo: assumo $2^m \leq (m+1)!$. Allora

$$2^{m+1} = 2 \cdot 2^m \leq 2 \cdot (m+1)! \leq (m+2)(m+1)! = (m+2)!$$

3.1 Induzione forte

Per dimostrare $P(m)$:

- assumo $P(k)$ vero $\forall\ k < m$;
- dimostro $P(m)$.

Allora $P(m)$ vale $\forall\ m$.

Nota 1: Utilità

L'induzione forte serve quando il passo induttivo richiede più casi precedenti, ad esempio per alberi binari e MergeSort.

3.2 Alberi binari

- **Altezza**: lunghezza del cammino più lungo dalla *radice* a una *foglia*.

Teorema 1

Per ogni albero binario T di altezza m, allora:

$$\#\text{foglie}(T) \leq 2^m$$

Dimostrazione per induzione su m:

- **Base** $m = 0$: l'albero è una sola foglia $\Rightarrow 1 \leq 2^0 = 1$.
- **Passo** $(P(m) \rightarrow P(m+1))$: sia T di altezza $m+1$. La radice ha due sottoalberi T' e T'' di altezze $m', m'' \leq m$. Allora (se uso *induzione completa*):

$$\#\text{foglie}(T) = \#\text{foglie}(T') + \#\text{foglie}(T'') \leq 2^{m'} + 2^{m''} \leq 2^m + 2^m = 2^{m+1}$$

4 Tempo di esecuzione

Per evitare di calcolarlo ogni volta si prendono le caratteristiche principali dell'algoritmo.

Prendiamo $f, g : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ con:

- $f(n)$ la funzione in esame della *complessità* del problema P;
- $g(n)$ è la funzione che moltiplicata per la costante c, dopo un certo n, fa da limite *superiore* o *inferiore* per ogni punto di $f(n)$.

Abbiamo quindi tre tipi di limite:

1. **Limite asintotico superiore** (O): $f(n) = O(g(n))$ se esiste $c > 0$ tale che $0 \leq f(n) \leq c \cdot g(n)$ per ogni $n \geq n_0$.
2. **Limite asintotico inferiore** (Ω): $f(n) = \Omega(g(n))$ se esiste $c > 0$ tale che $0 \leq c \cdot g(n) \leq f(n)$ per ogni $n \geq n_0$.
3. **Limite asintotico stretto** (Θ): $f(n) = \Theta(g(n))$ se esistono $c_1 > 0$ e $c_2 > 0$ tali che $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ per ogni $n \geq n_0$.

4.1 Metodo del limite

Date $f(n), g(n) > 0$:

1. Se $\lim\limits_{n\to\infty} \dfrac{f(n)}{g(n)} = k > 0$ e $k \neq \infty$, allora $f(n) = \Theta(g(n))$.
2. Se $\lim\limits_{n\to\infty} \dfrac{f(n)}{g(n)} = 0$, allora $f(n) = O(g(n))$ e $f(n) \neq \Omega(g(n))$.

3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$, allora $f(n) = \Omega(g(n))$ e $f(n) \neq O(g(n))$.

N.B.: non è valido il contrario; per esempio se $f(n) = \Theta(g(n))$ non è detto che il limite sia > 0 e $\neq \infty$.

Osservazioni:

- Se $f(n) = a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 = \Theta(n^k)$ allora
$$\lim_{n\to\infty} \frac{f(n)}{n^k} = a_k > 0.$$
- Se $\Theta(n^h) \neq \Theta(n^k)$ con $h \neq k$ allora
$$\lim_{n\to\infty} \frac{n^h}{n^k} = \infty \text{ o } 0.$$
- Se $\Theta(a^n) \neq \Theta(b^n)$ con $a \neq b$ e $a, b > 0$ allora
$$\lim_{n\to\infty} \frac{a^n}{b^n} = \lim_{n\to\infty} \left(\frac{a}{b}\right)^n = \begin{cases} 0 & a < b, \\ \infty & a > b. \end{cases}$$
- Se $\Theta(\log_a n) = \Theta(\log_b n)$ allora
$$\log_a n = (\log_a b) \cdot (\log_b n).$$

5 Complessità dei problemi

Definizione 2: Complessità

Dato un problema P, la **complessità** di P è la complessità dell'algoritmo *più efficiente* che risolve P.

Limite superiore: un algoritmo A che risolve P con complessità $O(g(n))$ determina un **limite superiore** $O(g(n))$ per la complessità di P.

Esempio 2: Limite superiore

Insertion Sort: $O(n^2)$
Merge Sort: $O(n \log n)$

Limite inferiore: se ogni algoritmo A che risolve P ha complessità $\Omega(g(n))$, allora P ha complessità $\Omega(g(n))$.

Esempio 3: Limite inferiore

Ordinamento: $\Omega(g(n))$ — devo almeno leggere gli elementi.

Limite stretto: se P è $\Omega(f(n))$ e $O(f(n))$ si dice che P è $\Theta(f(n))$.

6 Metodo di sostituzione

Può essere usato in alternativa al **Master Theorem**. Ha due fasi:

1. ipotesi di soluzione;
2. verifica che la soluzione è corretta per induzione.

Esempio 4: Metodo di sostituzione

$$T(n) = \begin{cases} 4 & \text{se } n = 1, \\ 2T\left(\frac{n}{2}\right) + 6n & \text{se } n > 1. \end{cases}$$

Ipotesi: considerando a, b, c opportune costanti, ipotizzo

$$T(n) = an \log_2 n + bn + c.$$

Dimostro per induzione che questa è la soluzione.

- Caso base ($n = 1$):
$$T(1) = a \cdot 1 \cdot \log_2 1 + b \cdot 1 + c = \boxed{b + c = 4}.$$
- Passo induttivo ($n > 1$): assumendo che $\forall m < n$, $T(m) = am \log_2 m + bm + c$, voglio dimostrare che vale per n:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + 6n \quad \left(\frac{n}{2} < n, \text{ uso ip. induttiva}\right) \\ &= 2\left(a\frac{n}{2}\log_2 \frac{n}{2} + b\frac{n}{2} + c\right) + 6n \\ &= an(\log_2 n - 1) + bn + 2c + 6n \\ &= an \log_2 n + (-a + b + 6)n + 2c. \end{aligned}$$

Voglio che valga

$$an \log_2 n + bn + c.$$

Serve quindi che:

$$\begin{aligned} a - b + 6 = b &\qquad \Rightarrow a = 6, \\ 2c = c &\qquad \Rightarrow c = 0, \\ b + c = 4,\ c = 0 &\qquad \Rightarrow b = 4. \end{aligned}$$

Soluzione:

$$\boxed{T(n) = 6n \log_2 n + 4n} \quad \Rightarrow \quad T(n) = \Theta(n \log n).$$

7 Master Theorem

Dato $T(n)$ che rappresenta il **tempo totale** dell'algoritmo dato un input n:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n),$$

con $a \geq 1$ il numero di sottoproblemi, $\frac{n}{b}$ con $b > 1$ la dimensione dei sottoproblemi, e $f(n)$ il costo della parte non ricorsiva.

Confrontando $f(n)$ con $n^{\log_b a}$, si hanno tre casi:

1. $T(n) = \Theta\left(n^{\log_b a}\right)$ se $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ per $\epsilon > 0$;
2. $T(n) = \Theta\left(n^{\log_b a} \log n\right)$ se $f(n) = \Theta\left(n^{\log_b a}\right)$;
3. $T(n) = \Theta\left(f(n)\right)$ se $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ per $\epsilon > 0$ e vale la *regolarità*: $\exists k$ con $0 < k < 1$ tale che $f\left(\frac{n}{b}\right) \leq kf(n)$ per ogni $n \geq n_0$.

7.1 Utilizzo

Con il *metodo del limite* per il confronto tra $f(n)$ e $n^{\log_b a}$:

- $\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a}} = k > 0,\ k \neq \infty \Rightarrow f(n) = \Theta(n^{\log_b a})$ (*caso 2*);

- $\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a}} = 0 \Rightarrow f(n) = O(n^{\log_b a})$ e $f(n) \neq \Omega(n^{\log_b a})$ (*caso 1*). Infine devo trovare $\epsilon > 0$ tale che

$$\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a - \epsilon}} = 0 \quad \Rightarrow \quad T(n) = \Theta(n^{\log_b a});$$

- $\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a}} = \infty \Rightarrow f(n) = \Omega(n^{\log_b a})$ e $f(n) \neq O(n^{\log_b a})$ (*caso 3*).

Esempio 5: Master Theorem applicato a MergeSort

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + a'n + b' \\ &= 2T\left(\frac{n}{2}\right) + \underbrace{a'n + b'}_{f(n)}. \end{aligned}$$

Dato che $a = 2$ e $b = 2$, allora $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. Quindi

$$\lim_{n\to\infty} \frac{a'n + b'}{n} > 0, \neq \infty.$$

Sono nel *caso 2* del Master Theorem, quindi

$$T(n) = \Theta(n \log n).$$

Nel caso non sia possibile risolvere con il Master Theorem si usa il metodo di sostituzione.

8 Insertion Sort Ricorsivo

8.1 Pseudocodice

```
InsertionSort(A, j):
    if j>1:
        InsertionSort(A, j-1)
        Insert(A,j)

Insert(A, j):
    if (j>1) and (A[j-1]>A[j]):
        A[j-1] <-> A[j]
        Insert(A,j-1)
```

8.2 Analisi

- Costo della funzione `Insert`:

$$T^{INS}(j) = c + T^{INS}(j-1) \quad \Rightarrow \quad T^{INS}(j) = \Theta(j)$$

- Costo funzione `InsertionSort`:

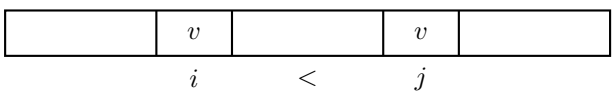
$$T^{IS}(n) = T^{IS}(n-1) + T^{INS}(n) + c$$

Quindi in totale

$$\boxed{T^{IS}(n) = T^{IS}(n-1) + \Theta(n) = \Theta\left(n^2\right)}$$

8.3 Duplicazione

Dato `A[1..n]` si dice duplicato:



Dove $i < j$ e `A[i]=A[j]`

Osservazione: `dup(A[p..r])=dup(A[p..q])` $\cup$ `dup(A[q+1,r])` $\cup$ `dupCross(A,p,q,r)`

Info 3: Idea

Per trovare i duplicati, arco-ordino l'array in modo simile al merge sort.
Metodo `CheckCrossDup(A,p,q,r)` che prende `A[p,q]`, `A[q+1,r]` ordinati e verifica la presenza di duplicati cross, se non ce ne sono ordina `A[p..r]`.

Per contare i duplicati:

$$|\{(i,j)|\ 1 \le i < j \le n\ \wedge\ \texttt{A[i]=A[j]}\}|$$

```
CountDup(A,p,r)
    if p<r
        q= (p+r)/2
        return CountDup(A,p,r)+CountDup(A,q+1,r)+CountCrossDup(A,p,q,r)
    else
        return 0;
```

9 HeapSort

Complessità

- tempo $\Theta(n \log n)$
- spazio $\Theta(1)$

Definizione 3: Albero Binario Completo

1. Ogni nodo non foglia ha solo due figli
2. Ogni cammino radice-foglia ha la stessa lunghezza

Quindi un albero è **completo** se ha la radice, con due figli e ognuno di essi è radice di un sotto albero completo della stessa altezza

Proposizione: Un albero completo di altezza h ha un numero di nodi $2^{n+1} - 1$.

Dimostrazione: Per induzione su h $\quad n(\mathrm{T}) =$ numero di nodi dell'albero T.

- Caso base $(h = 0)$:

$$n(\mathrm{T}) = 1 = 2^{0+1} - 1 \quad \checkmark$$

- Passo induttivo $(n > 0)$: $\quad$ Dove $h' = h - 1$, quindi:

$$n(\mathrm{T}) = n(\mathrm{T}_1) + n(\mathrm{T}_2) + 1 = 2\left(2^{h'+1} - 1\right) + 1 = 2(2^h - 1) + 1 = 2^{h+1} - 1 \quad \checkmark$$

Definizione 4: Albero binario quasi completo

Tutti i livelli sono completi tranne l'ultimo dove le foglie sono solo a sinistra.

Definizione 5: Heap

È un albero binario ordinato quasi completo implementato come array.
Ci sono due tipi di **Heap**:

- **Max Heap**: Dove ogni nodo `A[i]` è $\ge$ dei suoi figli, ma $\le$ dei suoi antenati/parenti
- **Min Heap**

10 QuickSort

Complessità:

- Caso peggiore: $O(n^2)$
- Caso medio: $O(n \log n)$

Info 4: Funzionamento

Come il mergeSort, applica il principio divide et impera:
- **Partiziona**: sceglie un elemento x detto Pivot in `A[p..r]`, adesso divide l'array in modo che ogni elemento di `A[p:q-1]` sia minore o uguale del pivot, mentre ogni elemento in `A[q+1:r]` è maggiore o uguale al pivot.
- **Impera**: ordina `A[p:q-1]` e `A[q+1:r]` ricorsivamente
- **Combina**: Non fa nulla, dato che gli array sono già ordinati, non serve combinarli

11 Esercizi

- 01/10/2026
- 05/10/2026